



CS336: Language Modeling from Scratch

Stanford / Spring 2025



The Spring 2024 offering of the course is [archived here](#).

Course Staff



Tatsunori Hashimoto
Instructor



Percy Liang
Instructor



Marcel Rød
CA



Neil Band
CA



Rohith Kuditipudi
CA

1.3 Transformer Language Model Architecture (Week 2)

- Token Embeddings: given a sequence of token IDs, the Transformer language model uses a token embedding layer to produce a sequence of vectors
- Relative Positional Embeddings
- Pre-norm Transformer Block:
- Output Normalization and Embedding
- Batching, Einsum and Efficient Computation
- Parameter Initialization: RMSNorm
- Root Mean Square Layer Normalization
- Position-Wise Feed-Forward Network
- Scaled Dot-Product Attention
- Causal Multi-Head Self-Attention

1.4 Training a Transformer LM (Week 3)

Cross-entropy loss

$$L = -\frac{1}{N} \left(\sum_{i=1}^N y_i \cdot \log(p_i) \right)$$

$$L = -\frac{1}{N} \left[\sum_{j=1}^N [t_j \log(p_j) + (1 - t_j) \log(1 - p_j)] \right]$$

```

import torch

def log_softmax_stable(logits: torch.Tensor) -> torch.Tensor:
    """
    Computes log(softmax(logits)) in a numerically stable way.

    Args:
        logits (torch.Tensor): Raw, unnormalized scores from a model.
                               Shape: (... , num_classes).

    Returns:
        torch.Tensor: The log-probabilities for each class.
                     Shape: (... , num_classes).
    """
    # 1. Subtract the max logit for numerical stability.
    max_logits = torch.max(logits, dim=-1, keepdim=True).values
    logits_shifted = logits - max_logits

    # 2. Compute the log of the sum of exponentials (the normalization term).
    log_sum_exp = torch.log(torch.exp(logits_shifted).sum(dim=-1, keepdim=True))

    # 3. The final log-probability is the shifted logit minus the normalization term.
    return logits_shifted - log_sum_exp

@torch.compile
def cross_entropy_loss(logits: torch.Tensor, targets: torch.Tensor) -> torch.Tensor:
    """
    Computes the cross-entropy loss from logits in a stable and modular way.

    Args:
        logits (torch.Tensor): Raw model output scores with shape
                               (... , num_classes).
        targets (torch.Tensor): Ground-truth labels (indices) with shape (...).

    Returns:
        torch.Tensor: A single scalar value representing the average loss.
    """
    # 1. Get the log-probabilities for all classes using the stable helper function.
    log_probs = log_softmax_stable(logits)

    # 2. Gather the log-probabilities of the true target classes.
    target_log_probs = torch.gather(log_probs, dim=-1, index=targets.unsqueeze(-1))

    # 3. Calculate the final loss.
    loss = -target_log_probs.mean()

    return loss

```

Tuning the learning rate

1. **Vanilla Gradient Descent (with Learning Rate):** The simplest update moves weights in the opposite direction of the gradient, scaled by the learning rate α .

$$W_{t+1} = W_t - \alpha \cdot \nabla J(W_t)$$

```

"""
A custom SGD optimizer to tune its learning rate.
"""

import math
from typing import Iterable, List

import torch
from torch.optim.optimizer import Optimizer

class SGD(Optimizer):
    """
    Implements a custom Stochastic Gradient Descent (SGD) optimizer.
    """

    def __init__(self, params: Iterable[torch.Tensor], lr: float = 1e-3):
        """
        Initializes the SGD optimizer.

        Args:
            params: An iterable of parameters to optimize.
            lr: The base learning rate (default: 1e-3).
        """
        # --- Input Arguments ---
        # params: An iterable of model parameters (e.g., model.parameters()).
        # lr: The learning rate, a float (e.g., 0.001).

        if lr < 0.0:
            raise ValueError(f"Invalid learning rate: {lr}")
        defaults = {"lr": lr}
        super().__init__(params, defaults)

    def step(self) -> None:
        """
        Performs a single optimization step.

        This method updates the parameters based on their gradients.
        """
        for group in self.param_groups:
            for p in group["params"]:
                if p.grad is None:
                    continue

                grad = p.grad.data
                state = self.state[p]
                t = state.get("t", 0)
                decayed_lr = group["lr"] / math.sqrt(t + 1)
                p.data.add_(grad, alpha=-decayed_lr)
                state["t"] = t + 1

        return

```

```

# =====
# Tuning Logic
# =====

def run_tuning_experiment(
    learning_rates: List[float],
    num_steps: int = 10
):
    """
    Runs a simple experiment to observe the effect of different learning rates.

    For each learning rate, it initializes a new set of weights and tracks
    the loss over several optimization steps.

    Args:
        learning_rates: A list of learning rates to test.
        num_steps: The number of optimization steps to run for each learning rate.
    """
    # --- Input Arguments ---
    # learning_rates: A list of floats to test (e.g., [1.0, 10.0]).
    # num_steps: The number of training steps for each test (e.g., 10).

    print("Starting Learning Rate Tuning Experiment...")
    for lr in learning_rates:
        print(f"\n" + "="*35)
        print(f"  Testing Learning Rate: {lr}")
        print("="*35)

        torch.manual_seed(42)
        weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
        optimizer = SGD([weights], lr=lr)

        for step in range(num_steps):
            optimizer.zero_grad()
            loss = (weights ** 2).mean()
            print(f"  Step {step + 1: >2}: Loss = {loss.item():.4f}")
            loss.backward()
            optimizer.step()

if __name__ == "__main__":
    # Define the learning rates you want to compare
    lrs_to_test = [1.0, 10.0, 100.0, 1000.0]

    # Run the experiment
    run_tuning_experiment(lrs_to_test)

```

As we will see, one of the hyperparameters that affects training the most is the learning rate. Let's see that in practice in our toy example. Run the SGD example above with three other values for the learning rate: 1e1, 1e2, and 1e3, for just 10 training iterations. What happens with the loss for each of these learning rates? Does it decay faster, slower, or does it diverge (i.e., increase over the course of training)?

Deliverable: A one-two sentence response with the behaviors you observed.

For learning rates of 10.0, 100.0, and 1000.0, the loss does not decay but instead diverges, increasing rapidly over the training iterations. This occurs because these learning rates are too high, causing the optimization steps to be so large that they repeatedly overshoot the minimum and result in exploding weight values.

AdamW optimizer

Let α be the learning rate, W_t the weights at step t , and g_t the gradient at step t .

- **SGD:** The basic update.

$$W_{t+1} = W_t - \alpha \cdot g_t$$

- **Momentum:** Introduces a velocity vector v_t . β is the momentum coefficient.

$$v_t = \beta_1 v_{t-1} + g_t$$

$$W_{t+1} = W_t - \alpha \cdot v_t$$

- **RMSProp:** Maintains a decaying average of past squared gradients s_t . β_2 is the decay rate.

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2$$

$$W_{t+1} = W_t - \frac{\alpha}{\sqrt{s_t} + \epsilon} \cdot g_t$$

- **Adam:** Combines both, maintaining a first moment (v_t) and a second moment (s_t), and performs bias correction (denoted with a hat, e.g., $\hat{v}_t$).

$$W_{t+1} = W_t - \frac{\alpha}{\sqrt{\hat{s}_t} + \epsilon} \cdot \hat{v}_t$$

- **Muon (based on provided sources):** Extends Adam by introducing a "decaying acceleration" mechanism that modulates momentum. It tracks gradient direction consistency to prevent overshooting, updating momentum based on a friction term ϕ_t .

$$\phi_t = \text{sigmoid}(\kappa \cdot (g_t \cdot v_{t-1}))$$

$$v_t = (\beta_1 \cdot \phi_t) v_{t-1} + (1 - \beta_1) g_t$$

$$W_{t+1} = W_t - \frac{\alpha}{\sqrt{\hat{s}_t} + \epsilon} \cdot \hat{v}_t$$

```

"""
A modular, PEP 8 compliant implementation of the AdamW optimizer.
"""

import math
from typing import Callable, Iterable, Optional, Tuple

import torch
from torch.optim.optimizer import Optimizer

class AdamW(Optimizer):
    """
    Implements the AdamW algorithm.
    """

    def __init__(
        self,
        params: Iterable[torch.Tensor],
        lr: float = 1e-3,
        betas: Tuple[float, float] = (0.9, 0.999),
        eps: float = 1e-8,
        weight_decay: float = 1e-2,
    ):
        """
        Initializes the AdamW optimizer.

        Args:
            params (Iterable[torch.Tensor]): An iterable of parameters to optimize.
            lr (float): The learning rate (default: 1e-3).
            betas (Tuple[float, float]): Coefficients for computing running
                averages of the gradient and its square (default: (0.9, 0.999)).
            eps (float): A term added to the denominator for numerical stability
                (default: 1e-8).
            weight_decay (float): The decoupled weight decay coefficient
                (default: 1e-2).
        """
        # --- Input Validation ---
        if not 0.0 <= lr:
            raise ValueError(f"Invalid learning rate: {lr}")
        if not 0.0 <= eps:
            raise ValueError(f"Invalid epsilon value: {eps}")
        if not 0.0 <= betas[0] < 1.0:
            raise ValueError(f"Invalid beta parameter at index 0: {betas[0]}")
        if not 0.0 <= betas[1] < 1.0:
            raise ValueError(f"Invalid beta parameter at index 1: {betas[1]}")
        if not 0.0 <= weight_decay:
            raise ValueError(f"Invalid weight_decay value: {weight_decay}")

        defaults = {
            "lr": lr,
            "betas": betas,
            "eps": eps,
            "weight_decay": weight_decay,
        }
        # The __init__ only needs to set up defaults. Unused instance attributes
        # like self.lr have been removed for clarity.
        super().__init__(params, defaults)

```

```

@torch.no_grad()
def step(self, closure: Optional[Callable] = None) -> Optional[torch.Tensor]:
    """
    Performs a single optimization step.

    Args:
        closure (Optional[Callable]): A callable that re-evaluates the model
            and returns the loss. This is optional.

    Returns:
        Optional[torch.Tensor]: The loss returned by the closure, if provided.
    """
    loss = None
    if closure is not None:
        loss = closure()

    for group in self.param_groups:
        # Retrieve hyperparameters for the current parameter group
        lr = group["lr"]
        beta1, beta2 = group["betas"]
        eps = group["eps"]
        weight_decay = group["weight_decay"]

        for p in group["params"]:
            if p.grad is None:
                continue

            grad = p.grad
            state = self.state[p]

            # --- State Initialization ---
            if len(state) == 0:
                state["step"] = 0
                # Exponential moving average of gradient values (m_t)
                state["exp_avg"] = torch.zeros_like(p)
                # Exponential moving average of squared gradient values (v_t)
                state["exp_avg_sq"] = torch.zeros_like(p)

            exp_avg, exp_avg_sq = state["exp_avg"], state["exp_avg_sq"]
            state["step"] += 1
            t = state["step"]

            # --- 1. Decoupled Weight Decay ---
            # This is the "W" in AdamW. It's a simple L2 penalty applied
            # directly to the weights, completely decoupled from the
            # gradient-based update.
            if weight_decay != 0:
                p.mul_(1 - lr * weight_decay)

            # --- 2. Update Biased Moment Estimates ---
            # Update the first moment (momentum)
            exp_avg.mul_(beta1).add_(grad, alpha=1 - beta1)
            # Update the second moment (variance)
            exp_avg_sq.mul_(beta2).addcmul_(grad, grad.conj(), value=1 - beta2)

            # --- 3. Compute Bias-Corrected Step Size ---
            # Correct for the bias introduced by initializing moments at zero
            bias_correction1 = 1 - beta1**t
            bias_correction2 = 1 - beta2**t
            step_size = lr * math.sqrt(bias_correction2) / bias_correction1

            # --- 4. Perform Adam Parameter Update ---
            # Calculate the denominator for the adaptive learning rate
            denom = exp_avg_sq.sqrt().add_(eps)
            p.addcdiv_(exp_avg, denom, value=-step_size)

    return loss

```

Problem (adamwAccounting): Resource accounting for training with AdamW (2 points)

Let us compute how much memory and compute running AdamW requires. Assume we are using float32 for every tensor.

(a) How much peak memory does running AdamW require? Decompose your answer based on the memory usage of the parameters, activations, gradients, and optimizer state. Express your answer in terms of the batch_size and the model hyperparameters (vocab_size, context_length, num_layers, d_model, num_heads). Assume $d_{ff} = 4 \times d_{model}$.

For simplicity, when calculating memory usage of activations, consider only the following components:

- Transformer block
 - RMSNorm(s)
 - Multi-head self-attention sublayer: QKV projections, $Q^T K$ matrix multiply, softmax, weighted sum of values, output projection.
 - Position-wise feed-forward: W_1 matrix multiply, SiLU, W_2 matrix multiply
- final RMSNorm
- output embedding
- cross-entropy on logits

Deliverable: An algebraic expression for each of parameters, activations, gradients, and optimizer state, as well as the total.

(b) Instantiate your answer for a GPT-2 XL-shaped model to get an expression that only depends on the batch_size. What is the maximum batch size you can use and still fit within 80GB memory?

Deliverable: An expression that looks like $a \cdot \text{batch_size} + b$ for numerical values a , b , and a number representing the maximum batch size.

(c) How many FLOPs does running one step of AdamW take?

Deliverable: An algebraic expression, with a brief justification.

(d) Model FLOPs utilization (MFU) is defined as the ratio of observed throughput (tokens per second) relative to the hardware's theoretical peak FLOP throughput [Chowdhery et al., 2022]. An NVIDIA A100 GPU has a theoretical peak of 19.5 teraFLOP/s for float32 operations. Assuming you are able to get 50% MFU, how long would it take to train a GPT-2 XL for 400K steps and a batch size of 1024 on a single A100? Following Kaplan et al. [2020] and Hoffmann et al. [2022], assume that the backward pass has twice the FLOPs of the forward pass.

Deliverable: The number of days training would take, with a brief justification.

(a) Peak Memory Requirement

The peak memory is the sum of memory for parameters, gradients, optimizer state, and activations. We assume **float32** (4 bytes per number). Let B be batch size, V be vocab size, T be context length, L be number of layers, D be model dimension, and H be number of heads. We also assume $d_{ff} = 4D$.

- **Parameters:** These are the model weights, which do not depend on the batch size. The dominant terms are the embedding layers and the weights within the Transformer blocks.

- Token & Positional Embeddings: $V \times D + T \times D$
- Per Transformer Layer (L layers total):
 - QKV & Output Projections: $4 \times D \times D$
 - FFN Layers: $D \times 4D + 4D \times D = 8D^2$
 - RMSNorm weights (gain only): $2 \times D$
- Final RMSNorm & unembedding layer: $D + D \times V$
- **Expression (Parameters):**

$$M_{params} = (V \cdot D + T \cdot D) + L \cdot (4D^2 + 8D^2 + 2D) + D + V \cdot D \approx \mathbf{12LD^2}$$

- **Gradients:** The memory required to store gradients is identical to the memory for parameters.

- **Expression (Gradients):**

$$M_{grads} = M_{params} \approx \mathbf{12LD^2 + 2VD}$$

- **Optimizer State:** AdamW stores two states for each parameter: the first and second moments (m and v), both in float32.

- **Expression (Optimizer State):**

$$M_{optim} = 2 \times M_{params} \approx \mathbf{24LD^2 + 4VD}$$

- **Activations:** This is the only component that scales with the batch size B . Peak activation memory is dominated by values that must be stored during the forward pass for use in the backward pass. The largest components are typically the attention score matrix, the output of the FFN's first layer, and the final logits. A standard approximation is used here.

- **Expression (Activations):**

$$M_{acts} \approx B \cdot T \cdot \left(L \cdot (2D + 2\frac{H}{D}T^2) + V \right)$$

- **Total Peak Memory (Bytes):** We sum all components and multiply by 4 bytes/float.

- **Expression (Total):**

$$M_{total} = 4 \times [(M_{params} + M_{grads} + M_{optim}) + M_{acts}]$$

$$M_{total} = 4 \times \left[(1 + 1 + 2) \cdot (12LD^2 + 2VD) + B \cdot T \cdot \left(L \cdot (2D + 2\frac{H}{D}T^2) + V \right) \right]$$

$$\mathbf{M_{total} = 4 \times \left[4(12LD^2 + 2VD) + B \cdot T \cdot \left(L \cdot (2D + 2\frac{H}{D}T^2) + V \right) \right]}$$

(b) Instantiation for GPT-2 XL

For a GPT-2 XL model, the hyperparameters are: $L = 48$, $D = 1600$, $H = 25$, $V \approx 50,000$, and $T = 1024$.

- Model State Memory (the b term):** This is the memory for parameters, gradients, and the optimizer state, which is independent of the batch size.
 - Number of Parameters $\approx 12LD^2 + 2VD = 12 \cdot 48 \cdot 1600^2 + 2 \cdot 50000 \cdot 1600 \approx 1.47 \times 10^9 + 0.16 \times 10^9 \approx 1.63 \times 10^9$. We'll use the known value of **1.5B**.
 - Memory = $4 \times (1.5 \times 10^9 \text{ params}) \times (1 \text{ for params} + 1 \text{ for grads} + 2 \text{ for optim state}) = 16 \times 1.5 \times 10^9 = \mathbf{24 \times 10^9}$ bytes, or **24 GB**.
- Activation Memory (the a term):** This is the memory per batch item.
 - $a = 4 \times [T \cdot (L \cdot (2D + 2\frac{H}{D}T^2) + V)]$
 - $a = 4 \times [1024 \cdot (48 \cdot (2 \cdot 1600 + 2 \cdot \frac{25}{1600} \cdot 1024^2) + 50000)]$
 - $a = 4 \times [1024 \cdot (48 \cdot (3200 + 32768) + 50000)]$
 - $a = 4 \times [1024 \cdot (1.72 \times 10^6 + 50000)] \approx 4 \times [1.8 \times 10^9] = \mathbf{7.2 \times 10^9}$ bytes, or **7.2 GB**.
- Final Expression and Max Batch Size:**
 - Expression:** The total memory is $M_{total} \approx (7.2 \cdot \text{batch_size} + 24) \text{ GB}$.
 - To fit within 80 GB of memory:
$$7.2 \cdot B + 24 \leq 80$$
$$7.2 \cdot B \leq 56$$
$$B \leq 7.77$$
 - Maximum Batch Size:** The maximum integer batch size is **7**.

(c) FLOPs per AdamW Step

The total FLOPs per step is the sum of the forward pass, backward pass, and optimizer step. We assume the optimizer step FLOPs are negligible and that **Backward FLOPs $\approx 2 \times$ Forward FLOPs**. Therefore, **Total FLOPs $\approx 3 \times$ Forward FLOPs**.

The forward pass FLOPs are dominated by matrix multiplications. A standard and widely used approximation is that the forward pass takes approximately **$2 \times \text{NumParams}$** FLOPs per token.

- Expression:**

$$\text{FLOPs}_{step} \approx 3 \times (\text{FLOPs}_{fwd})$$

$$\text{FLOPs}_{step} \approx 3 \times (2 \cdot \text{NumParams} \cdot B \cdot T)$$

$$\text{FLOPs}_{step} \approx \mathbf{6 \cdot NumParams \cdot B \cdot T}$$

Justification: This approximation accounts for the matmuls in the attention and FFN layers, which dominate the computation.

(d) Training Time on a Single A100

1. Calculate FLOPs per step for GPT-2 XL with B=1024:
- NumParams (P) = 1.5×10^9 .

• $\text{FLOPs}_{step} \approx 6 \cdot (1.5 \times 10^9) \cdot 1024 \cdot 1024 \approx \mathbf{9.45 \times 10^{15}}$ FLOPs.
2. Calculate Total FLOPs for Training:
- Total FLOPs = 400,000 steps $\times$ (9.45×10^{15} FLOPs/step) = $\mathbf{3.78 \times 10^{21}}$ FLOPs.
3. Calculate Effective GPU Throughput:
- An A100 has a peak throughput of 19.5 TFLOP/s = 19.5×10^{12} FLOP/s.

• With 50% MFU, the effective throughput is $0.50 \times 19.5 \times 10^{12} = \mathbf{9.75 \times 10^{12}}$ FLOP/s.
4. Calculate Total Training Time:
- Time (seconds) = $\frac{\text{Total FLOPs}}{\text{Effective Throughput}} = \frac{3.78 \times 10^{21}}{9.75 \times 10^{12}} \approx \mathbf{3.88 \times 10^8}$ seconds.

• Time (days) = $\frac{3.88 \times 10^8 \text{ s}}{86400 \text{ s/day}} \approx \mathbf{4490}$ days.

Deliverable: Training would take approximately **4490 days** on a single A100 GPU. This highlights why training large models requires large-scale, parallel clusters of hundreds or thousands of accelerators.

Learning rate scheduling

```
"""
PyTorch hyperparameter scheduling.
"""

import math
from typing import Any, Callable

from torch.optim import Optimizer

# =====
# 1: Core Decay Functions
# =====

def _get_linear_value(start_val: float, end_val: float, progress: float) -> float:
    """Linearly interpolates between a start and end value."""
    return start_val + progress * (end_val - start_val)

def _get_cosine_value(start_val: float, end_val: float, progress: float) -> float:
    """Interpolates between a start and end value using a cosine curve."""
    cos_val = 0.5 * (1.0 + math.cos(math.pi * progress))
    return end_val + (start_val - end_val) * cos_val

def _get_exp_value(start_val: float, end_val: float, progress: float) -> float:
    """Interpolates between a start and end value using an exponential curve."""
    if start_val <= 0:
        return 0.0
    return start_val * (end_val / start_val) ** progress
```

```

# =====
# 2: Generic Multi-Phase Scheduler
# =====

DECAY_REGISTRY = {
    "linear": _get_linear_value,
    "cosine": _get_cosine_value,
    "exp": _get_exp_value,
}

def get_multiphase_schedule_value(
    it: int,
    warmup_iters: int,
    initial_value: float,
    schedule: list[dict],
) -> float:
    """
    Calculates a value based on a generic multi-phase schedule with warmup.

    This single function can handle LR, sequence length, or any other
    hyperparameter scheduling by replacing the original, duplicated functions.

    Args:
        it: The current iteration step.
        warmup_iters: Number of linear warmup steps from 0 to `initial_value`.
        initial_value: The value after warmup / at the start of the first phase.
        schedule: A list of dicts defining the schedule's phases. Each dict
                  requires 'until_iter', 'to_val', and 'type'.

    """
    # Phase 1: Handle linear warmup
    if it < warmup_iters:
        progress = it / max(1, warmup_iters)
        return _get_linear_value(0.0, initial_value, progress)

    # Phase 2: Find the current decay phase and calculate the value
    phase_start_iter = warmup_iters
    phase_start_value = initial_value

    for phase in schedule:
        phase_end_iter = phase["until_iter"]
        phase_end_value = phase["to_val"]

        if it <= phase_end_iter:
            decay_type = phase.get("type", "linear")
            if decay_type not in DECAY_REGISTRY:
                raise ValueError(f"Unknown decay type: '{decay_type}'")

            decay_func = DECAY_REGISTRY[decay_type]
            decay_steps = phase_end_iter - phase_start_iter
            progress = (it - phase_start_iter) / max(1, decay_steps)

            return decay_func(phase_start_value, phase_end_value, progress)

        phase_start_iter = phase_end_iter
        phase_start_value = phase_end_value

    # Phase 3: Return the final value after all phases are complete
    return schedule[-1]["to_val"]

```

```
# =====
# 3: Specialized Scheduler
# =====

def get_lr_with_cosine_cycle(
    it: int, lr_max: float, lr_min: float, warmup_iters: int, cycle_iters: int
) -> float:
    """Calculates LR for a single cosine decay cycle with warmup."""
    schedule = [{"until_iter": cycle_iters, "to_val": lr_min, "type": "cosine"}]
    return get_multiphase_schedule_value(it, warmup_iters, lr_max, schedule)

def get_lr_with_double_cycle(
    it: int,
    lr_max: float,
    lr_inter: float,
    lr_min: float,
    warmup_iters: int,
    phase_one_iters: int,
    phase_two_iters: int,
    phase_two_type: str = "cosine",
) -> float:
    """Calculates LR for a two-phase decay schedule (exp -> linear/cosine)."""
    schedule = [
        {"until_iter": phase_one_iters, "to_val": lr_inter, "type": "exp"},
        {"until_iter": phase_two_iters, "to_val": lr_min, "type": phase_two_type},
    ]
    return get_multiphase_schedule_value(it, warmup_iters, lr_max, schedule)
```

```
# =====
# 4: Stateful Scheduler Class
# =====

class HyperparameterScheduler:
    """
    A stateful class to apply any scheduling function to an optimizer.

    This class is now generic and decoupled from the actual scheduling logic.
    """

    def __init__(
        self,
        optimizer: Optimizer,
        scheduling_func: Callable[..., float],
        param_name: str = "lr",
        **schedule_kwargs: Any,
    ):
        """
        Initializes the hyperparameter scheduler.

        Args:
            optimizer: The PyTorch optimizer to update.
            scheduling_func: A stateless function (like the ones defined above)
                that takes `it` as its first argument and returns the value.
            param_name: The name of the hyperparameter in the optimizer's
                param_groups to update (e.g., 'lr').
            **schedule_kwargs: Keyword arguments to pass to the scheduling_func.
        """
        if not isinstance(optimizer, Optimizer):
            raise TypeError(f"{type(optimizer).__name__} is not an Optimizer")
        if not callable(scheduling_func):
            raise TypeError("scheduling_func must be a callable function")

        self.optimizer = optimizer
        self.scheduling_func = scheduling_func
        self.param_name = param_name
        self.schedule_kwargs = schedule_kwargs
        self.it = -1
        self.current_value = 0.0
        self.reset()

    def step(self) -> None:
        """Advances the scheduler by one step and updates the optimizer."""
        self.it += 1
        self.current_value = self.scheduling_func(it=self.it, **self.schedule_kwargs)

        for param_group in self.optimizer.param_groups:
            param_group[self.param_name] = self.current_value

    def reset(self) -> None:
        """Resets the scheduler to its initial state before the first step."""
        self.it = -1
        self.step()
```

Gradient clipping

```
"""A module for a custom gradient clipping utility in PyTorch."""

from collections.abc import Iterable
from typing import List

import torch

def _compute_total_grad_norm(grads: List[torch.Tensor]) -> torch.Tensor:
    """
    Computes the total L2 norm of a list of gradient tensors. 🧠

    Args:
        grads: A list of gradient tensors.

    Returns:
        A single scalar tensor containing the total L2 norm.
    """
    # Sum the squared L2 norm of each gradient tensor
    total_norm_sq = sum(g.detach().pow(2).sum() for g in grads)
    return torch.sqrt(total_norm_sq)

@torch.compile
def clip_grad_norm_(
    parameters: Iterable[torch.nn.Parameter], max_norm: float
) -> torch.Tensor:
    """
    Clips the total L2 norm of gradients for a set of parameters in-place.

    Args:
        parameters: An iterable of model parameters. Parameters with `None`
            gradients are skipped.
        max_norm: The maximum allowed L2 norm for the gradients.

    Returns:
        A tensor containing the total L2 norm of the parameters' gradients
        (before clipping).
    """
    # Filter out parameters that do not have gradients
    grads = [p.grad for p in parameters if p.grad is not None]

    if not grads:
        return torch.tensor(0.0)

    # 1. Compute the total L2 norm using the modular helper function.
    total_norm = _compute_total_grad_norm(grads)

    # 2. Apply clipping if the norm exceeds the maximum.
    if total_norm > max_norm:
        # Calculate the scaling factor to shrink the norm to max_norm.
        scale = max_norm / (total_norm + 1e-6)
        # Apply the scaling to all gradients in-place.
        for grad in grads:
            grad.detach().mul_(scale)

    return total_norm
```

1.5 Training loop (Week 3)

Data Loader

```
import numpy as np
import torch

class DataLoader:
    """
    data loader for language model pre-training.

    Usage:
        loader = DataLoader(data, batch_size=32, context_length=256, device="cuda")
        for x_batch, y_batch in loader:
            # Your training step here
        ...
    """

    def __init__(
        self,
        data: np.ndarray,
        batch_size: int,
        context_length: int,
        device: str
    ):
        """
        Args:
            data (np.ndarray): The 1D numpy array of token IDs.
            batch_size (int): The number of sequences in each batch.
            context_length (int): The length of each token sequence.
            device (str): The device to move the tensors to (e.g., 'cpu', 'cuda').
        """
        self.data = data
        self.batch_size = batch_size
        self.context_length = context_length
        self.device = device

        # Pre-calculate the maximum valid starting index for a sequence
        self.max_start_idx = len(data) - context_length - 1
        if self.max_start_idx < 0:
            raise ValueError(
                f"Data length ({len(data)}) is too short for the given "
                f"context length ({context_length})."
            )

    def _to_device(self, x: torch.Tensor, y: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
        """Helper method to move a batch to the target device."""
        if self.device == "cuda":
            # Pin memory for faster GPU transfer
            return (
                x.pin_memory().to(self.device, non_blocking=True),
                y.pin_memory().to(self.device, non_blocking=True),
            )
        return x.to(self.device), y.to(self.device)

    def get_batch(self) -> tuple[torch.Tensor, torch.Tensor]:
        """
        Generates a single, random batch of (x, y) data.

        Returns:
            A tuple containing the input batch (x) and the target batch (y).
        """
        # 1. Sample 'batch_size' random starting indices all at once.
        start_indices = np.random.randint(0, self.max_start_idx + 1, size=self.batch_size)

        # 2. Create a matrix of indices for all sequences in the batch.
        # This is the core vectorization trick. It creates a (batch_size, context_length)
        # matrix of indices to select all batch data in a single operation.
        # np.arange creates the offsets [0, 1, ..., context_length - 1]
        # Broadcasting adds this to the start_indices column vector.
        x_indices = start_indices[:, np.newaxis] + np.arange(self.context_length)
        y_indices = x_indices + 1 # Targets are shifted by one position

        # 3. Slice the data array using the index matrices to create the batches.
        x_sequences = self.data[x_indices]
        y_sequences = self.data[y_indices]

        # 4. Convert numpy arrays to PyTorch tensors.
        x_batch = torch.from_numpy(x_sequences.astype(np.int64))
        y_batch = torch.from_numpy(y_sequences.astype(np.int64))

        # 5. Move the batch to the specified device using the helper method.
        return self._to_device(x_batch, y_batch)

    def __iter__(self):
        """Allows the class to be used as an iterator."""
        return self

    def __next__(self):
        """Defines the behavior of the iterator for the next item."""
        return self.get_batch()
```

Checkpointing (save_checkpoint/load_checkpoint)

```
import os
from typing import IO, BinaryIO, List, Optional, Union

import torch
import torch.nn as nn
import torch.optim as optim

# =====
#  Helper Functions
#  =====

def _unwrap_model(model: nn.Module) -> nn.Module:
    """
    Unwraps a model from potential wrappers like `torch.compile` or DDP.
    """
    # Handles models optimized with torch.compile()
    unwrapped = getattr(model, "_orig_mod", model)
    # Handles models wrapped in DistributedDataParallel or FSDP
    module = getattr(unwrapped, "module", unwrapped)
    return module

def _get_optimizer_list(
    optimizers: Union[optim.Optimizer, List[optim.Optimizer]]
) -> List[optim.Optimizer]:
    """
    Ensures the optimizers argument is always a list for consistent handling.
    """
    return [optimizers] if isinstance(optimizers, optim.Optimizer) else optimizers
```

```

def save_checkpoint(
    model: nn.Module,
    optimizers: Union[optim.Optimizer, List[optim.Optimizer]],
    iteration: int,
    out: Union[str, os.PathLike, BinaryIO, IO[bytes]],
):
    """
    Saves a training checkpoint to a file or buffer. 📁

    Args:
        model (nn.Module): The model to save the state of. It will be
            automatically unwrapped if compiled or distributed.
        optimizers (Union[optim.Optimizer, List[optim.Optimizer]]): One or
            more optimizers whose state will be saved.
        iteration (int): The current training iteration or step.
        out (Union[str, os.PathLike, ...]): The path or file-like object
            where the checkpoint will be saved.
    """
    # Use helper functions for clarity
    unwrapped_model = _unwrap_model(model)
    optimizer_list = _get_optimizer_list(optimizers)

    # Construct the state dictionary
    state_dict = {
        "model": unwrapped_model.state_dict(),
        "optimizers": [opt.state_dict() for opt in optimizer_list],
        "iteration": iteration,
    }

    torch.save(state_dict, out)


def load_checkpoint(
    src: Union[str, os.PathLike, BinaryIO, IO[bytes]],
    model: Optional[nn.Module] = None,
    optimizers: Optional[Union[optim.Optimizer, List[optim.Optimizer]]] = None,
) -> int:
    """
    Loads a training checkpoint from a file or buffer.

    The model and optimizers are updated in-place.

    Args:
        src (Union[str, os.PathLike, ...]): The path or file-like object
            to load the checkpoint from.
        model (Optional[nn.Module]): The model to load the state into.
            If None, model state is not loaded.
        optimizers (Optional[Union[...]]): One or more optimizers to load
            state into. If None, optimizer state is not loaded.

    Returns:
        int: The iteration number from the loaded checkpoint to resume training.
    """
    # Load checkpoint onto CPU first to avoid GPU memory issues
    checkpoint = torch.load(src, map_location="cpu")

    # Load model state if a model is provided
    if model is not None:
        unwrapped_model = _unwrap_model(model)
        unwrapped_model.load_state_dict(checkpoint["model"])

    # Load optimizer states if optimizers are provided
    if optimizers is not None:
        optimizer_list = _get_optimizer_list(optimizers)
        for optimizer, state_dict in zip(optimizer_list, checkpoint["optimizers"]):
            optimizer.load_state_dict(state_dict)

    # Return the iteration number to allow for resuming training
    return checkpoint["iteration"]

```

training_together

```
# config.py
import os
import json
import yaml
import argparse
import time
import torch
from typing import Any, Dict, Optional

class Config(Dict):
    """A config object that allows attribute-style access to dictionary keys."""
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.__dict__ = self
        for key, value in self.items():
            if isinstance(value, dict):
                self[key] = Config(value)

def _load_config_from_file(config_path: str) -> Dict[str, Any]:
    """Loads configuration from a YAML or JSON file."""
    ext = os.path.splitext(config_path)[1].lower()
    with open(config_path, 'r') as f:
        if ext == ".json":
            return json.load(f)
        elif ext in [".yaml", ".yml"]:
            return yaml.safe_load(f)
        else:
            raise ValueError(f"Unsupported config file format: {ext}")

def _deep_set(config_dict, key_path: str, value):
    """Deeply sets a dot-separated key in a dictionary."""
    keys = key_path.split(".")
    d = config_dict
    for k in keys[:-1]:
        d = d.setdefault(k, {})
    d[keys[-1]] = value

def get_config() -> Config:
    """
    Parses command-line arguments, loads, and merges configurations.

    This function orchestrates the entire configuration setup, including
    default values, user-specified files, and command-line overrides.

    Returns:
        Config: The final, merged configuration object.
    """
    parser = argparse.ArgumentParser(description="Train a transformer model.")
    parser.add_argument("--config", type=str, help="Path to a custom config file.")
    parser.add_argument("--override", action="append", default=[], help="Override a conf
    args = parser.parse_args()

    # Load default configuration
    default_config_path = os.path.join(os.path.dirname(__file__), "configs/default.yml")
    config_dict = _load_config_from_file(default_config_path)

    # Override with user-specified config file
    if args.config:
        user_config = _load_config_from_file(args.config)
        config_dict.update(user_config)

    # Apply command-line overrides
    for override_str in args.override:
        key, value = override_str.split("=", 1)
        _deep_set(config_dict, key, value)

    config = Config(config_dict)

    # --- Set runtime properties ---
    config.run.run_id = config.run.run_id.replace("<timestamp>", f"{int(time.time())}")

    if torch.cuda.is_available():
        config.device = "cuda"
        config.dtype = "bfloat16"
    else:
        config.device = "cpu"
        config.dtype = "float32"

    return config
```

```

# trainer.py
import os
import time
import torch
import torch.nn as nn
import numpy as np
import wandb

from config import Config
from cs336_basics.model import Transformer
from cs336_basics.data_loader import DataLoader
from cs336_basics.loss import cross_entropy_loss
from cs336_basics.lr_schedule import lr_cosine_schedule
from cs336_basics.checkpointing import CheckpointManager
from cs336_basics.gradient_clip import clip_grad_norm_

class Trainer:
    """
    A class to encapsulate the model training loop. 🚀
    """
    def __init__(self, config: Config, model: Transformer, optimizer: torch.optim.Optimizer,
                  train_loader: DataLoader, valid_loader: DataLoader):
        self.config = config
        self.model = model
        self.optimizer = optimizer
        self.train_loader = train_loader
        self.valid_loader = valid_loader
        self.device = config.device
        self.dtype = getattr(torch, config.dtype)

        self.run_dir = os.path.join(config.run.out_dir, config.run.run_id)
        self.checkpoint_manager = CheckpointManager(os.path.join(self.run_dir, "checkpoints"))
        self._setup_logging()

    def _setup_logging(self):
        """Initializes logging with W&B and sets up directories."""
        os.makedirs(self.run_dir, exist_ok=True)
        self.wandb_run = None
        if self.config.run.wandb_project:
            self.wandb_run = wandb.init(
                project=self.config.run.wandb_project,
                id=self.config.run.run_id,
                name=self.config.run.run_id,
                config=self.config,
                dir=self.run_dir,
            )
        print(f"Run outputting to: {self.run_dir}")

    def _get_lr(self, step: int) -> float:
        """Calculates the learning rate for the current step."""
        # This can be expanded to include the other LR schedules
        return lr_cosine_schedule(
            it=step,
            lr_max=self.config.training.lr_max,
            lr_min=self.config.training.lr_min,

```

```

# main.py
import numpy as np
import torch
from config import get_config
from trainer import Trainer
from cs336_basics.model import Transformer
from cs336_basics.data_loader import DataLoader
from cs336_basics.adamw import AdamW

def main():
    """
    Main entrypoint for training the Transformer model.
    """

    # 1. Get Configuration
    config = get_config()
    torch.manual_seed(config.run.seed)

    # 2. Load Data
    train_data = np.memmap(config.data.train_data_path, dtype=np.uint16, mode='r')
    valid_data = np.memmap(config.data.valid_data_path, dtype=np.uint16, mode='r')

    train_loader = DataLoader(
        data=train_data,
        batch_size=config.training.batch_size,
        context_length=config.model.context_length,
        device=config.device
    )
    valid_loader = DataLoader(
        data=valid_data,
        batch_size=config.training.batch_size,
        context_length=config.model.context_length,
        device=config.device
    )

    # 3. Initialize Model
    model = Transformer(**config.model)
    model.to(config.device)
    print(f"Model initialized with {sum(p.numel() for p in model.parameters()):,} parameters")

    # 4. Initialize Optimizer
    optimizer = AdamW(model.parameters(), **config.optimizer)

    # 5. Initialize and Run Trainer
    trainer = Trainer(
        config=config,
        model=model,
        optimizer=optimizer,
        train_loader=train_loader,
        valid_loader=valid_loader
    )
    trainer.train()

if __name__ == "__main__":
    main()

```

1.6 Generating text (Week 3)

Generating text (Decoding)

```

import torch
import torch.nn.functional as F
from cs336_basics.tokenizer import Tokenizer
from cs336_basics.model import Transformer

# =====
#  Helper Function
#  =====

def _sample_from_logits(
    logits: torch.Tensor, temperature: float, top_p: float
) -> int:
    """
    Samples a token ID from logits using temperature and nucleus (top-p) sampling.
    """

    # 1. Apply temperature scaling to soften or sharpen the distribution
    if temperature > 0:
        logits = logits / temperature

    # 2. Convert logits to probabilities using the softmax function
    probs = F.softmax(logits, dim=-1)

    # 3. top-p sampling
    sorted_probs, sorted_indices = torch.sort(probs, descending=True)
    cumulative_probs = torch.cumsum(sorted_probs, dim=-1)

    # Create a mask to remove tokens outside the nucleus
    sorted_indices_to_remove = cumulative_probs > top_p
    # Shift the mask to the right to ensure we keep the first token that
    # exceeds the top_p threshold, which completes the nucleus.
    sorted_indices_to_remove[..., 1:] = sorted_indices_to_remove[..., :-1].clone()
    sorted_indices_to_remove[..., 0] = 0

    # Apply the mask to the original, unsorted probabilities
    indices_to_remove = sorted_indices[sorted_indices_to_remove]
    probs[indices_to_remove] = 0.0

    # 4. Re-normalize the filtered probabilities so they sum to 1
    renormalized_probs = probs / probs.sum()

    # 5. Sample from the final, re-normalized distribution
    next_token_id = torch.multinomial(renormalized_probs, num_samples=1)

    return next_token_id.item()

```

```

@torch.no_grad()
def decode(
    model: Transformer,
    tokenizer: Tokenizer,
    prompt: str,
    max_new_tokens: int = 32,
    temperature: float = 0.7,
    top_p: float = 0.9,
) -> str:
    """
    Generates text autoregressively from a Transformer model.
    """

    # Set model to evaluation mode
    model.eval()

    # Initial setup
    input_ids = tokenizer.encode(prompt)
    end_id = tokenizer.encode("<|endoftext|>")[0]
    context_length = model.context_length
    device = next(model.parameters()).device

    # The main autoregressive generation loop
    for _ in range(max_new_tokens):
        # 1. Manage the context window to prevent it from exceeding model limits
        current_ids = input_ids[-context_length:]
        x = torch.tensor([current_ids], dtype=torch.long, device=device)

        # 2. Get model predictions (logits) for the next token
        logits = model(x)
        next_token_logits = logits[0, -1, :] # We only need the logits for the last token

        # 3. Sample the next token using the modular helper function
        next_token_id = _sample_from_logits(next_token_logits, temperature, top_p)

        # 4. Check for the end-of-text token to stop generation
        if next_token_id == end_id:
            break

        # 5. Append the new token to the sequence and continue the loop
        input_ids.append(next_token_id)

    # Restore model to training mode and decode the final sequence
    model.train()
    return tokenizer.decode(input_ids)

```